

TECHNICAL REPORT

Expression Solver

JavaFX Desktop Application

Infix → Postfix Conversion & Evaluation

Using Shunting-Yard Algorithm with Stack-Based Evaluation

Date: May 13, 2026

Platform: JavaFX 17+ | Language: Java

1. Introduction

This report presents a comprehensive technical analysis of the Expression Solver, a JavaFX desktop application that evaluates mathematical infix expressions at runtime. The application converts infix notation to postfix (Reverse Polish Notation) using the classic Shunting-Yard algorithm and then evaluates the resulting postfix expression using a stack-based calculator.

1.1 Purpose and Scope

The Expression Solver was designed to demonstrate core data-structures concepts — particularly stack-based algorithms — within a polished, themed graphical interface. It accepts arithmetic expressions containing integers and floating-point numbers, the four basic operators, exponentiation, and parentheses for grouping.

1.2 Key Features

- Real-time infix-to-postfix conversion with step-by-step trace output
- Stack-based postfix evaluation supporting $+$, $-$, $\times$, $\div$, and $^$ operators

- Ocean-themed JavaFX UI with responsive layout and keyboard shortcut (Enter key)
- Detailed per-token conversion trace displayed in a scrollable text area
- Robust error handling for unbalanced parentheses, division by zero, and malformed tokens

2. System Architecture

The application follows a single-class architecture (`HelloApplication`) that extends `javafx.application.Application`. All UI construction, event wiring, and algorithmic logic are encapsulated within this class. The design prioritises clarity and self-containment over multi-module separation, making it well-suited for an educational demonstration context.

2.1 Component Overview

Component	Type	Responsibility
<code>start(Stage)</code>	JavaFX entry point	Builds and wires the complete scene graph
<code>convertToPostfix()</code>	Core algorithm	Shunting-yard: produces postfix string & trace log
<code>computePostfix()</code>	Core algorithm	Stack evaluator: reduces postfix to a numeric result
<code>splitTokens()</code>	Utility	Tokenises infix string via regex spacing + split
UI builder helpers	UI factory methods	<code>makeLabel</code> , <code>buildButton</code> , <code>buildTile</code> , <code>applyFieldStyle</code> , etc.

2.2 Data Flow

The application processes input through three sequential stages:

1. User types an infix expression (e.g., $(8+2)*3-1$) into the `TextField` and presses Solve or Enter.
2. `splitTokens()` tokenises the string; `convertToPostfix()` applies the Shunting-Yard algorithm and emits a postfix string plus a formatted trace log.
3. `computePostfix()` evaluates the postfix string and returns a double; the result is formatted and displayed in the Answer tile.

3. Core Algorithms

3.1 Tokenisation

Before either algorithm runs, the raw input string is normalised by `splitTokens()`:

```
expr.replaceAll("([+\\-*/^()])", " $1 ").trim().split("\\s+")
```

The regex inserts spaces around every operator and bracket, then splits on whitespace. This allows the user to enter expressions with or without spaces and still receive correct results (e.g., "(8+2)*3-1" is equivalent to "(8 + 2) * 3 - 1").

3.2 Infix-to-Postfix Conversion (Shunting-Yard)

The `convertToPostfix` method implements E. W. Dijkstra's Shunting-Yard algorithm. It iterates over tokens and applies the following rules:

Token Type	Action
Number / operand	Append directly to the output queue.
Left parenthesis (	Push onto the operator stack.
Right parenthesis)	Pop operators to output until a matching (is found; discard both parentheses. Throw if no (is found.
Operator	While the stack top is a non-(operator with priority $\geq$ current operator's priority, pop to output; then push the current operator.
End of input	Pop all remaining operators to output; throw if any (remains.

Operator priorities: $^ (3) > * / (2) > + - (1)$. These are implemented in the `priority()` helper method via a `switch` expression.

Worked Example

Input: (8+2)*3-1

Token	Action	Stack	Output
(	push	()	
8	output	()	8
+	push	(+)	8
2	output	(+)	8 2
)	pop until (	empty	8 2 +
*	push	*	8 2 +
3	output	*	8 2 + 3
-	pop *, push -	-	8 2 + 3 *
1	output	-	8 2 + 3 * 1
(end)	pop -	empty	8 2 + 3 * 1 -

Result: 8 2 + 3 * 1 -

3.3 Postfix Evaluation

computePostfix() processes the postfix string using a second stack of doubles:

- Each numeric token is parsed and pushed onto the stack.
- Each operator pops two operands (right, then left), applies the operation, and pushes the result.
- After all tokens, the stack must contain exactly one value — the final answer.

Continuing the example above, evaluation of "8 2 + 3 * 1 -" proceeds:

Token	Action	Stack State
8	push 8	[8]
2	push 2	[8, 2]
+	pop 2 & 8, push 10	[10]
3	push 3	[10, 3]
*	pop 3 & 10, push 30	[30]
1	push 1	[30, 1]
-	pop 1 & 30, push 29	[29]

Final answer: 29

4. User Interface Design

4.1 Theme and Visual Palette

The application employs a deep-ocean colour scheme to create a professional, focused aesthetic. All colours are declared as static string constants at the top of the class, enabling consistent theming across every widget.

Constant	Hex Value	Usage
DEEP_BG	#0b1622	Root window background
PANEL_BG	#132238	Left and right panel backgrounds
TILE_BG	#1b3050	Result tiles, text field, trace area
CYAN_ACC	#00c8e8	Banner text, postfix value, field border accent
GOLD_ACC	#f0a830	Section headers within result tiles
LIGHT_TXT	#d8e8f4	Primary body text on dark backgrounds
DIM_TXT	#5888a0	Tagline, prompt text, subtle borders
ERR_CLR	#f06060	Validation error messages
OK_CLR	#50e898	Numeric answer display

4.2 Layout Structure

The root layout is a VBox containing four regions stacked vertically:

- Top bar (HBox): Application banner and tagline, left-aligned.
- Separator: A thin horizontal rule providing visual section division.
- Mid section (HBox): Left input panel and right results panel, side by side with equal growth weights.
- Bottom panel (VBox): Scrollable conversion trace area spanning the full width.

4.3 Interaction Design

- The Solve button triggers the complete algorithm pipeline and updates all three output areas atomically.
- The Reset button clears the input field and resets all output labels to placeholder state.
- Pressing Enter in the expression field fires the Solve action via `exprField.setOnAction`, eliminating the need to move focus to the button.
- Both buttons have hover opacity effects (`setOnMouseEntered` / `Exited`) that provide visual feedback.
- The window is non-resizable (`setResizable(false)`), ensuring the fixed layout always renders correctly.

5. Error Handling

All errors are caught in the solve button's event handler and surfaced through the `errMsg` label displayed beneath the input buttons. The output tiles are reset to “...” and the trace area is cleared to avoid showing stale data alongside an error message.

Error Condition	Detection Point	Message
Empty input	Solve button handler	⚠ Enter an expression first!
Unmatched) parenthesis	<code>convertToPostfix()</code>	Unbalanced parentheses.
Unmatched (in stack at end	<code>convertToPostfix()</code>	Unbalanced parentheses.
Unrecognised character	<code>convertToPostfix()</code>	Unrecognised token: [char]
Fewer than 2 operands for operator	<code>computePostfix()</code>	Malformed expression.
Division by zero	<code>computePostfix()</code>	Cannot divide by zero.
Extra operands remain on stack	<code>computePostfix()</code>	Malformed expression.

6. Identified Code Issue

A syntax error exists in the provided source at line 48, inside the `start()` method:

```
Label label = new Label("Type your expression below");
label.set    // ← INCOMPLETE – method call not finished; will not compile
```

The label variable is created but the subsequent method call on it is truncated. This line must be removed or completed before the project will compile. The correctly named prompt label declared on the line above already fulfils the intended purpose; the redundant label declaration should be deleted entirely.

7. Test Cases

Input Expression	Expected Postfix	Expected Answer	Category
$(8+2)*3-1$	8 2 + 3 * 1 -	29	Parentheses
2^{10}	2 10 ^	1024	Exponentiation
$10/4$	10 4 /	2.5000	Float result
$3+4*2$	3 4 2 * +	11	Precedence
$10/0$	10 0 /	ERROR	Div by zero
$(3+5$	N/A	ERROR	Unbalanced (
$3+5)$	N/A	ERROR	Unbalanced)
$((1+2)*3)/4$	1 2 + 3 * 4 /	2.2500	Nested parens

8. Conclusion

The Expression Solver successfully demonstrates two fundamental computer-science algorithms — Shunting-Yard and stack-based postfix evaluation — within a well-structured, visually polished JavaFX application. The separation between algorithmic logic and UI construction is clear, and the helper-method pattern keeps the codebase maintainable.

The one blocking defect (the incomplete label.set call) must be resolved before compilation. Beyond that, the application is functionally complete for its intended scope. Potential future enhancements include:

- Support for unary minus (e.g., -5 or -(3+2)).
- Support for mathematical functions such as sin(), cos(), sqrt().
- History panel recording previously evaluated expressions.
- Input validation with real-time highlighting of mismatched parentheses.